

SDP Codec API Specification

Version 1.0; July 5, 2004

SDP

NOKIA

Copyright © 2004 Nokia Corporation. All rights reserved.

Nokia and Nokia Connecting People are registered trademarks of Nokia Corporation. Java and all Java-based marks are trademarks or registered trademarks of Sun Microsystems, Inc. Other product and company names mentioned herein may be trademarks or trade names of their respective owners.

Disclaimer

The information in this document is provided “as is,” with no warranties whatsoever, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of any proposal, specification, or sample. Furthermore, information provided in this document is preliminary, and may be changed substantially prior to final release. This document is provided for informational purposes only.

Nokia Corporation disclaims all liability, including liability for infringement of any proprietary rights, relating to implementation of information presented in this document. Nokia Corporation does not warrant or represent that such use will not infringe such rights.

Nokia Corporation retains the right to make changes to this specification at any time, without notice.

License

A license is hereby granted to download and print a copy of this specification for personal use only. No other license to any other intellectual property rights is granted herein.

Contents

1	Introduction.....	6
1.1	List of Required Interfaces	6
1.2	Product Platform and/or Symbian OS Version Constraints.....	6
2	Technical Specification.....	7
2.1	Type of Interface.....	7
2.2	Interface Class Structure	7
2.3	Usage.....	7
2.3.1	Protocol	7
2.3.2	Error handling	9
2.3.3	Memory overhead.....	9
2.3.4	Extensions to the API.....	9
2.3.5	Example.....	9
2.4	Detailed Description	12
2.4.1	CSdpDocument.....	12
2.4.2	CSdpOriginField.....	24
2.4.3	CSdpConnectionField.....	33
2.4.4	CSdpBandwidthField	41
2.4.5	CSdpTimeField.....	45
2.4.6	TSdpTypedTime	50
2.4.7	CSdpRepeatField	51
2.4.8	CSdpKeyField.....	56
2.4.9	CSdpAttributeField.....	60
2.4.10	CSdpFmtAttributeField	64
2.4.11	TSdpRtpmapValue.....	68
2.4.12	CSdpMediaField.....	70
2.4.13	CSdpCodecStringPool	82
2.4.14	Code Architecture.....	84
3	Index	85
3.1	List of Constants and Enums.....	85
3.2	Alphabetical List of Classes and Methods	89
3.2.1	CSdpAttributeField	89
3.2.2	CSdpBandwidthField	89
3.2.3	CSdpCodecStringPool	89
3.2.4	CSdpConnectionField.....	89
3.2.5	CsdpDocument	90
3.2.6	CSdpFmtAttributeField	90

3.2.7	CSdpKeyField.....	91
3.2.8	CSdpMediaField.....	91
3.2.9	CSdpOriginField.....	92
3.2.10	CSdpRepeatField	92
3.2.11	CSdpTimeField.....	93
3.2.12	TSdpRtpmapValue.....	93
3.2.13	TSdpTypedTime	93
4	Terms and Abbreviations	94
5	References	95

Change History

July 5, 2004	Version 1.0	Initial document release

1 Introduction

When initiating multimedia teleconferences, voice-over-IP calls, streaming video, or other real-time sessions, there is a requirement to convey media details, transport addresses, and other session description metadata to the participants. SDP provides a standard representation for such information, irrespective of how that information is transported. Transport protocols that can contain an SDP description include SAP, SIP, RTSP, electronic mail using the MIME extensions, and HTTP. [1], [2], [3], and [4].

The SDP Codec subsystem provides services for converting SDP descriptions to the internal representation format and vice versa. The SDP Codec subsystem offers its clients an API for encoding/decoding SDP descriptions as well as setting and getting SDP field values.

1.1 List of Required Interfaces

None.

1.2 Product Platform and/or Symbian OS Version Constraints

This API is valid for all the platforms running Symbian OS v7.0s.

2 Technical Specification

2.1 Type of Interface

The logical type of the SDP Codec API is Library API. All function calls are synchronous.

2.2 Interface Class Structure

The client uses some or all SDP Codec API classes during the handling of an SDP description. All interface classes are inherited from the `Cbase` class. The client has a use relationship to the SDP Codec API library.

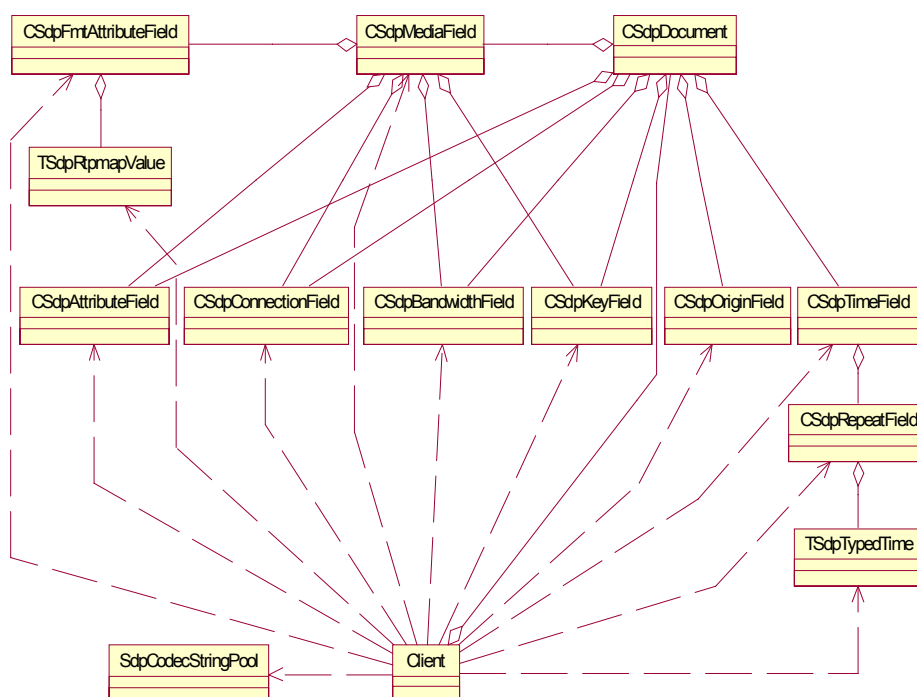


Figure 1: SDPCodec class structure

2.3 Usage

2.3.1 Protocol

The usage of the SDP Codec is depicted in the sequence diagram in Figure 2. The diagram includes only mandatory SDP fields. The media part of the SDP description is also included in the example.

Figure 2 depicts how the SDP description is created into a structural form. The first client creates the `CSdpDocument` instance. The string pool must be opened before it is used. All predefined SDP constants are listed in the string table, whose handle can be queried with the `SdpCodecStringPool::StringPoolL()` function. The predefined connection and origin field parameters `net type` and `address type` are received from the string pool. The origin field is created with origin field parameters and set into the `CSdpDocument` instance. The connection field is created with parameters and set into the `CSdpDocument` instance. When handling the SDP description ends, `CSdpDocument` is deleted and the string pool is closed.

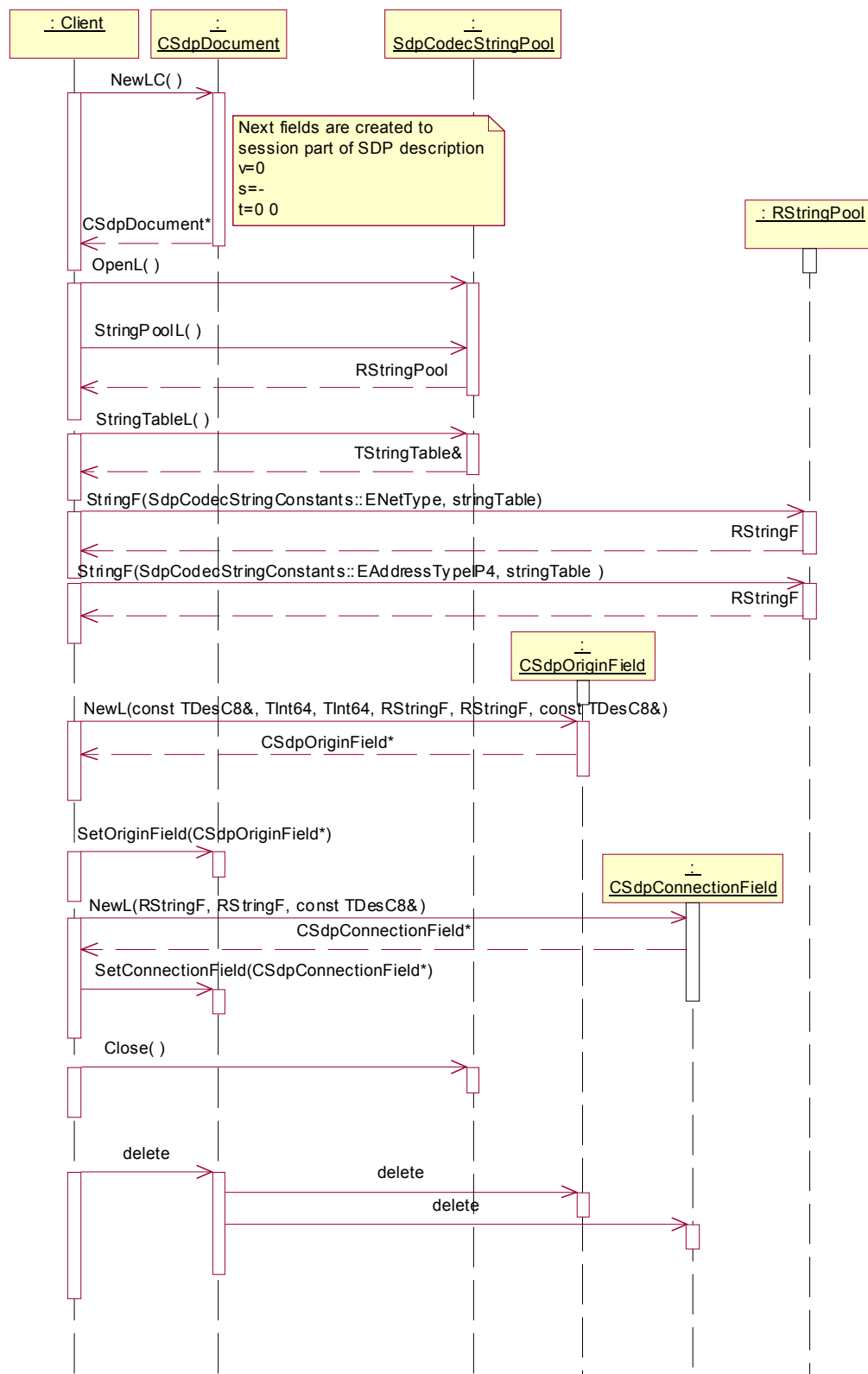


Figure 2: SDP description structural creation with all mandatory fields

As mentioned above, the SDP Codec is a library API. `CSdpDocument` and `CSdpStringPool` are the classes which are usually instantiated first. Other classes are instantiated when SDP fields corresponding to those classes are created.

2.3.2 Error handling

All function calls of this API are synchronous. The function leaves in case of an error.

The SDP Codec -specific error codes are listed in Section 3.1, "List of Constants and Enums." Symbian OS error codes are also used.

2.3.3 Memory overhead

The amount of overhead memory used by every SDP Codec API field object is at least the same as the memory used by the SDP field as text.

2.3.4 Extensions to the API

There are no possible extensions to this API.

2.3.5 Example

The following code example shows how to create an SDP description with mandatory fields.

Create the first `CSdpDocument` instance, which will include all SDP fields in structural format.

```
CSdpDocument* sdpDocument = CSdpDocument::NewLC();
```

Define the session name field for the SDP description.

```
sdpDocument->SetSessionNameL(_L8("SipSession"));
```

Create the origin field class instance and set it to the SDP description.

```
TInt64 sessionId( TInt( 2890844526 ) );
TInt64 sessionVersion( TInt( 2890842807 ) );
TInetAddr address;
const TInt32 KInetAddr = INET_ADDR(10,47,16,5);
address.SetAddress( KInetAddr );
CSdpOriginField* originfield = CSdpOriginField::NewL(
_L8("username"), sessionId, sessionversion, address);
sdpDocument->SetOriginField(originfield);
```

The string pool must be opened before it can be used. All predefined SDP constants are in the string table, which can be accessed from the string pool. When handling the SDP description is ended, the string pool must be closed.

```
SdpCodecStringPool::OpenL();
RStringPool pool = SdpCodecStringPool::StringPoolL();
```

The predefined network type and address type for the connection field are received from the string pool.

```
RStringF netType = pool.StringF( SdpCodecStringConstants::ENetType,
SdpCodecStringConstants::Table );
```

```
RStringF addressTypeIP4 = pool.StringF(
SdpCodecStringConstants::EAddressTypeIP4, SdpCodecStringConstants::Table
);
```

Create the connection field and set it to the SDP description.

```
_LIT8( KAddress, "10.47.16.5" );

CSdpConnectionField* connectionfield = CSdpConnectionField::NewL(
netType, addressTypeIP4, KAddress );

sdpDocument->SetConnectionField(connection);
```

Define the session part attribute field is defined for the SDP description. Because the `direction` attribute is not one of the predefined attributes in the string table, it must be defined dynamically to the string pool.

```
_LIT8(KSDPAttributeDirectionBoth,"both");

RStringF modifier = pool.OpenFStringL(_L8("direction"));

CleanupClosePushL(modifier);

CSdpAttributeField* attrField = CSdpAttributeField::NewL(modifier,
KSDPAttributeDirectionBoth);

User::LeaveIfError((sdpDocument->AttributeFields()).Append(attrField));

CleanupStack::PopAndDestroy();
```

Create the first media field.

```
RStringF mediaVideo = pool.StringF(
SdpCodecStringConstants::EMediaVideo,
SdpCodecStringPool::StringTableL());
RStringF protocol =
pool.StringF(SdpCodecStringConstants::EProtocolRtpAvp,
SdpCodecStringPool::StringTableL());
CSdpMediaField* medial = CSdpMediaField::NewLC(mediaVideo, 49152,
protocol, _L8("96"));
```

Add `sendonly` attribute ("a=sendonly") to the media.

```
RStringF sendonlyAttr = iPool.StringF( SdpCodecStringConstants::
EAttributeSendonly, SdpCodecStringPool::StringTableL() );
CSdpAttributeField* attr1 = CSdpAttributeField::NewLC(sendonlyAttr,
_L8(""));
User::LeaveIfError((medial->AttributeFields()).Append(attr1));
CleanupStack::Pop(); //attr1
```

Add `rtpmap` attribute ("a=rtpmap:96 H263-2000/90000") to the media.

```
_LIT8( KEncodingName, "H263-2000" );
_LIT8( KClockRate, "90000" );
_LIT8( KEncodingParam, "" );

TSdpRtpmapValue rtpmapValue( KEncodingName(), KClockRate(),
KEncodingParam() );
HBufC8* rtpmapBuf = rtpmapValue.EncodeL();
CleanupStack::PushL(rtpmapBuf);

RStringF rtpmapStr = pool.StringF(
SdpCodecStringConstants::EAttributeRtpmap,
SdpCodecStringPool::StringTableL() );
```

```

CSdpFmtAttributeField* rtpmapAttribute =
CSdpFmtAttributeField::NewLC(rtpmapStr, _L8("96"), *rtpmapBuf);

User::LeaveIfError(media1->FormatAttributeFields().Append
(rtpmapAttribute));
CleanupStack::Pop(); // rtpmapAttribute
CleanupStack::PopAndDestroy(); // rtpmapBuf

```

Create the second media.

```

RStringF mediaAudio = pool.StringF( SdpCodecStringConstants::
EMediaAudio, SdpCodecStringPool::StringTableL());

CSdpMediaField* media2 = CSdpMediaField::NewLC(mediaAudio, 57344,
protocol, _L8("97"));

```

Create the sendonly attribute for the second media.

```

CSdpAttributeField* attr2 = CSdpAttributeField::NewLC(sendonlyAttr,
_L8(""));

User::LeaveIfError((media2->AttributeFields()).Append(attr2));

CleanupStack::Pop();

```

Create the rtpmap attribute ("a=rtpmap:97 AMR/8000") for the second media.

```

_LIT8( KEncodingName1, "AMR" );
_LIT8( KClockRate1, "8000" );

TSdpRtpmapValue rtpmapValue1( KEncodingName1(), KClockRate1(),
KEncodingParam() );

HBufC8* rtpmapBuf1 = rtpmapValue1.EncodeL();

CleanupStack::PushL(rtpmapBuf1);

CSdpFmtAttributeField* rtpmapAttribute1 =
CSdpFmtAttributeField::NewLC(rtpmapStr, _L8("97"), *rtpmapBuf1);

User::LeaveIfError(media2->FormatAttributeFields().Append
(rtpmapAttribute1));

CleanupStack::Pop(); // rtpmapAttribute1
CleanupStack::PopAndDestroy(); //rtpmapBuf1

```

Put media fields to the CSdpDocument instance and remove the buffers from the cleanup stack.

```

User::LeaveIfError(doc->MediaFields().Append(media1));
User::LeaveIfError(doc->MediaFields().Append(media2));

CleanupStack::Pop(); //media2
CleanupStack::Pop(); //media1

```

Close the string pool and remove and delete the CSdpDocument instance from the cleanup stack.

```

SdpCodecStringPool::Close();

CleanupStack::PopAndDestroy(); //sdpDocument

```

The SDP description created in the previous example includes the following fields:

```

v=0

o=username 2890844526 2890842807 IN IP4 10.47.16.5

```

```

s=SipSession
c=IN IP4 10.47.16.5
t=0 0
a=direction:both
m=video 49152 RTP/AVP 96
a=sendonly
a=rtpmap:96 H263-2000/90000
m=audio 57344 RTP/AVP 97
a=sendonly
a=rtpmap:97 AMR/8000

```

2.4 Detailed Description

This section sets out the detailed specification of all SDP Codec API classes and their functions.

2.4.1 CSdpDocument

This class encapsulates the Session Description Protocol (SDP) document. The SDP document is a collection of SDP fields and related values describing a session. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.1.1 *DecodeL*

Function	<code>static CSdpDocument* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new SDP document.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted SDP document.
Return value	<code>CSdpDocument*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.2 *DecodeLC*

Function	<code>static CSdpDocument* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new SDP document and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted SDP document.
Return value	<code>CSdpDocument*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the case function leaves.

2.4.1.3 *NewL*

Function	<code>static CSdpDocument* NewL()</code>
Description	Constructs a new, empty SDP document. The instance will have the following initial values: <code>v=0</code> <code>s=-</code> <code>t=0 0.</code>
Preconditions	None
Parameters	None
Return value	<code>CSdpDocument*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.4 *NewLC*

Function	<code>static CSdpDocument* NewLC()</code>
Description	Constructs a new empty SDP document and adds the pointer to the cleanup stack. The instance will have the following initial values: <code>v=0</code> <code>s=-</code> <code>t=0 0.</code>
Preconditions	None
Parameters	None
Return value	<code>CSdpDocument*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.5 ~CsdpDocument

Function	<code>~CsdpDocument()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.1.6 ExternalizeL

Function	<code>void ExternalizeL(RWriteStream& aStream) const</code>
Description	Externalizes the object state in an internal, memory-optimized format.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : The output stream.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.7 InternalizeL

Function	<code>static CSdpDocument* InternalizeL(RReadStream& aStream);</code>
Description	Internalizes the object state from an internal, memory-optimized format.
Preconditions	None
Parameters	<code>RReadStream& aStream</code> : The input stream.
Return value	<code>CSdpDocument*</code> : A new SDP document internalized from the input stream.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.8 EncodeL

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the SDP document formatted according to the SDP syntax. It is possible to output the document even though it is incomplete in the sense that all mandatory SDP fields are not present. When creating an SDP document to be used, for example, as part of SIP signalling, the caller should first ensure the validity of the document with the <code>IsValid()</code> function.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted SDP description.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.9 CloneL

Function	<code>CSdpDocument* CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpDocument*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.10 IsValid

Function	<code>TBool IsValid() const</code>
Description	Checks if the instance represents a valid SDP document.
Preconditions	None
Parameters	None
Return value	<code>TBool: ETrue</code> if the SDP document is valid and <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.1.11 SdpVersion

Function	<code>TUint SdpVersion() const</code>
Description	Gets the SDP version used in the version field. Note that the current implementation supports only version number 0.
Preconditions	None
Parameters	None
Return value	<code>TUint</code> : The SDP version number.
Postconditions	None
Exceptions	-

2.4.1.12 OriginField

Function	<code>CSdpOriginField* OriginField()</code>
Description	Gets a reference to the session origin field. Note that the ownership is not transferred and the instance must not be deleted by the caller.
Preconditions	None
Parameters	None
Return value	<code>CSdpOriginField*</code> : The origin field or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.13 OriginField

Function	<code>const CSdpOriginField* OriginField() const</code>
Description	Gets a reference to the session origin field.
Preconditions	None
Parameters	None
Return value	<code>const CSdpOriginField*</code> : The origin field or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.14 SetOriginField

Function	<code>void SetOriginField(CSdpOriginField* aObj)</code>
Description	Sets or removes the session origin field. Note that the ownership of the argument is transferred to the SDP document object.
Preconditions	None
Parameters	<code>CSdpOriginField* aObj</code> : The new origin field or null to remove the field.
Return value	None
Postconditions	None
Exceptions	-

2.4.1.15 SessionName

Function	<code>const TDesC8& SessionName() const</code>
Description	Gets the session name field value.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The session name or empty descriptor if not present.
Postconditions	None
Exceptions	-

2.4.1.16 SetSessionNameL

Function	<code>void SetSessionNameL(const TDesC8& aValue)</code>
Description	Sets the session name field value.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : Valid session name value. Note that an empty value is not accepted.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecSessionField</code> if <code>aValue</code> is not empty or <code>aValue</code> includes invalid byte strings (<code>'\0'</code> , <code>'\n'</code> , <code>'\r'</code>).

2.4.1.17 Info

Function	<code>const TDesC8& Info() const</code>
Description	Gets the session information field value.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : Info field value or an empty descriptor if not present.
Postconditions	None
Exceptions	-

2.4.1.18 SetInfoL

Function	<code>void SetInfoL(const TDesC8& aValue)</code>
Description	Sets or removes the session information field value.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : New field value or an empty descriptor to remove the field.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecInfoField</code> if <code>aValue</code> is not empty or <code>aValue</code> includes invalid byte strings (<code>'\0'</code> , <code>'\n'</code> , <code>'\r'</code>).

2.4.1.19 Uri

Function	<code>CUri8* Uri()</code>
Description	Gets the session URI field value. Note that the ownership is not transferred and the instance must not be deleted by the caller.
Preconditions	None
Parameters	None
Return value	<code>CUri8*</code> : Session URI or zero if the field is not present.
Postconditions	None
Exceptions	-

2.4.1.20 Uri

Function	<code>const CUri8* Uri() const</code>
Description	Gets the session URI field value.
Preconditions	None
Parameters	None
Return value	<code>const CUri8*</code> : Session URI or zero if the field is not present.
Postconditions	None
Exceptions	-

2.4.1.21 SetUri

Function	<code>void SetUri(CUri8* aValue)</code>
Description	Sets or removes the session level URI field. Note that the ownership of the argument is transferred to the SDP document object.
Preconditions	None
Parameters	<code>CUri8* aValue</code> : New value of the field or zero to remove the field.
Return value	None
Postconditions	None
Exceptions	-

2.4.1.22 EmailFields

Function	CDesC8Array& EmailFields()
Description	Gets the set of session level e-mail field values. This array is used to add fields to and remove fields from the document. Note that only correctly formatted e-mail field values should be placed into the array. Empty values in the array will be ignored.
Preconditions	None
Parameters	None
Return value	CDesC8Array&: E-mail fields in array or empty array if no e-mail fields exist.
Postconditions	None
Exceptions	-

2.4.1.23 PhoneFields

Function	CDesC8Array& PhoneFields()
Description	Gets the set of session level phone field values. This array is used to add and remove phone fields into the document. Note that only correctly formatted phone field values should be placed into the array. Empty values in the array will be ignored.
Preconditions	None
Parameters	None
Return value	CDesC8Array&: Phone fields in array or empty array if no phone fields exist.
Postconditions	None
Exceptions	-

2.4.1.24 ConnectionField

Function	CSdpConnectionField* ConnectionField()
Description	Gets the session level connection field. Note that the ownership is not transferred and the instance must not be deleted by the caller.
Preconditions	None
Parameters	None
Return value	CSdpConnectionField*: Connection field or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.25 *ConnectionField*

Function	<code>const CSdpConnectionField* ConnectionField() const</code>
Description	Gets the session level connection field.
Preconditions	None
Parameters	None
Return value	<code>const CSdpConnectionField*</code> : Connection field or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.26 *SetConnectionField*

Function	<code>void SetConnectionField(CSdpConnectionField* aObj)</code>
Description	Sets or removes the session level connection field. Note that the ownership of the argument is transferred to the SDP document object.
Preconditions	None
Parameters	<code>CSdpConnectionField* aObj</code> : New connection field or zero to remove the field.
Return value	None
Postconditions	None
Exceptions	-

2.4.1.27 *BandwidthFields*

Function	<code>RPointerArray<CSdpBandwidthField>& BandwidthFields()</code>
Description	Gets the possibly empty set of session level bandwidth fields. This array is used directly for element insertion and removal. The array may contain zero references and the references are ignored. The objects referenced from the array are owned by the document instance and must not be deleted. An object can be removed from the document by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpBandwidthField>&</code> : Set of bandwidth fields.
Postconditions	None
Exceptions	-

2.4.1.28 TimeFields

Function	<code>RPointerArray<CSdpTimeField>& TimeFields()</code>
Description	Gets the set of session level time description fields. This array is used directly for element insertion and removal. There must be at least one time description field in a valid SDP document. The array may contain zero references and the references are ignored. The objects referenced from the array are owned by the document instance and must not be deleted. An object can be removed from the document by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpTimeField>&</code> : Set of time fields.
Postconditions	None
Exceptions	-

2.4.1.29 ZoneAdjustments

Function	<code>const TDesC8& ZoneAdjustments() const</code>
Description	Gets the zone adjustments field value.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The field value or an empty descriptor if the field is not present.
Postconditions	None
Exceptions	-

2.4.1.30 SetZoneAdjustmentsL

Function	<code>void SetZoneAdjustmentsL(const TDesC8& aValue)</code>
Description	Sets the zone adjustments field value.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : A valid field value or an empty descriptor if field is not present.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.1.31 Key

Function	<code>CSdpKeyField* Key()</code>
Description	Gets the session level encryption key field. Note that the ownership is not transferred and the instance must not be deleted by the caller.
Preconditions	None
Parameters	None
Return value	<code>CSdpKeyField*</code> : The encryption key or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.32 Key

Function	<code>const CSdpKeyField* Key() const</code>
Description	Gets the session level encryption key field.
Preconditions	None
Parameters	None
Return value	<code>const CSdpKeyField*</code> : The encryption key or zero if not present.
Postconditions	None
Exceptions	-

2.4.1.33 SetKey

Function	<code>void SetKey(CSdpKeyField* aObj)</code>
Description	Sets or removes the session level encryption key field. Note that the ownership of the argument is transferred to the SDP document object.
Preconditions	None
Parameters	<code>CSdpKeyField* aObj</code> : New value of the field or zero to remove the field.
Return value	None
Postconditions	None
Exceptions	-

2.4.1.34 AttributeFields

Function	<code>RPointerArray<CSdpAttributeField>& AttributeFields()</code>
Description	Gets the possibly empty set of session level attribute fields. This array is used directly for element insertion and removal. The array may contain zero references and the references are ignored. The objects referenced from the array are owned by the document instance and must not be deleted. An object can be removed from the document by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpAttributeField>&</code> : Set of session level attributes.
Postconditions	None
Exceptions	-

2.4.1.35 MediaFields

Function	<code>RPointerArray<CSdpMediaField>& MediaFields()</code>
Description	Gets the possibly empty set of media description fields. This array is used directly for element insertion and removal. Note that media level attributes and fields are managed through the corresponding media field instance and not through the document instance. The array may contain zero references and the references are ignored. The objects referenced from the array are owned by the document instance and must not be deleted. An object can be removed from the document by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpMediaField>&</code> : Set of media description fields.
Postconditions	None
Exceptions	-

2.4.2 CSdpOriginField

This class encapsulates the origin field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.2.1 DecodeL

Function	<code>static CSdpOriginField* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new origin field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.2 DecodeLC

Function	<code>static CSdpOriginField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new origin field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.3 NewL

Function	<code>static CSdpOriginField* NewL(const TDesC8& aUserName, TInt64 aSessionId, TInt64 aSessionVersion, TInetAddr& aAddress)</code>
Description	Constructs a new origin field.
Preconditions	None
Parameters	<code>const TDesC8& aUserName</code> : A valid user name value. <code>TInt64 aSessionId</code> : A valid session ID value. <code>TInt64 aSessionVersion</code> : A valid announcement version value. <code>TInetAddr& aAddress</code> : A unicast IP address from either <code>KAfInet</code> or <code>KAfInet6</code> family.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.4 NewL

Function	<code>static CSdpOriginField* NewL(const TDesC8& aUserName, TInt64 aSessionId, TInt64 aSessionVersion, RStringF aNetType, RStringF aAddressType, const TDesC8& aAddress)</code>
Description	Constructs a new origin field.
Preconditions	None
Parameters	<code>const TDesC8& aUserName</code>: A valid user name value. <code>TInt64 aSessionId</code>: A valid session id value. <code>TInt64 aSessionVersion</code>: A valid announcement version value. <code>RStringF aNetType</code>: A valid network type value (typically "IN"). <code>RStringF aAddressType</code>: A valid address type value (typically "IP4" or "IP6"). <code>const TDesC8& aAddress</code>: A unicast IP address, Fully Qualified Domain Name (FQDN), or another allowed address type.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.5 NewLC

Function	<code>static CSdpOriginField* NewLC(const TDesC8& aUserName, TInt64 aSessionId, TInt64 aSessionVersion, TInetAddr& aAddress)</code>
Description	Constructs a new origin field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aUserName</code>: A valid user name value. <code>TInt64 aSessionId</code>: A valid session id value. <code>TInt64 aSessionVersion</code>: A valid announcement version value. <code>TInetAddr& aAddress</code>: A unicast IP address from either <code>KAfInet</code> or <code>KAfInet6</code> family.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.6 *NewLC*

Function	<code>static CSdpOriginField* NewLC(const TDesC8& aUserName, TInt64 aSessionId, TInt64 aSessionVersion, RStringF aNetType, RStringF aAddressType, const TDesC8& aAddress)</code>
Description	Constructs a new origin field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aUserName</code>: A valid user name value. <code>TInt64 aSessionId</code>: A valid session ID value. <code>TInt64 aSessionVersion</code>: A valid announcement version value. <code>RStringF aNetType</code>: A valid network type value (typically "IN"). <code>RStringF aAddressType</code>: A valid address type value (typically "IP4" or "IP6"). <code>const TDesC8& aAddress</code>: A unicast IP address, Fully Qualified Domain Name (FQDN), or another allowed address type.
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.7 *~CSdpOriginField*

Function	<code>~CSdpOriginField()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.2.8 *EncodeL*

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the field formatted according to the SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. When returned, the stream includes the correctly formatted origin field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.9 *CloneL*

Function	<code>CSdpOriginField* CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpOriginField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.2.10 *operator==*

Function	<code>TBool operator == (const CSdpOriginField & aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpOriginField & aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.2.11 UserName

Function	<code>const TDesC8& UserName() const</code>
Description	Gets the user name of the origin field.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : User name.
Postconditions	None
Exceptions	-

2.4.2.12 SetUserNameL

Function	<code>void SetUserNameL(const TDesC8& aValue)</code>
Description	Sets the user name.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : A valid user name value.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecOriginField</code> if <code>aValue</code> is not a valid user name.

2.4.2.13 SessionId

Function	<code>TInt64 SessionId() const</code>
Description	Gets the session ID of the origin field.
Preconditions	None
Parameters	None
Return value	<code>TInt64</code> : Session ID
Postconditions	None
Exceptions	-

2.4.2.14 SetSessionId

Function	void SetSessionId(TInt64 aValue)
Description	Sets the session ID.
Preconditions	None
Parameters	TInt64 aValue: A valid session ID value.
Return value	None
Postconditions	None
Exceptions	-

2.4.2.15 Version

Function	TInt64 Version() const
Description	Gets the announcement version.
Preconditions	None
Parameters	None
Return value	TInt64: Version.
Postconditions	None
Exceptions	-

2.4.2.16 SetVersion

Function	void SetVersion(TInt64 aValue)
Description	Sets the announcement version.
Preconditions	None
Parameters	TInt64 aValue: A valid version value.
Return value	None
Postconditions	None
Exceptions	-

2.4.2.17 NetType

Function	<code>RStringF NetType() const</code>
Description	Gets the net type.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The net type from predefined SDP string table or given by the user.
Postconditions	None
Exceptions	-

2.4.2.18 AddressType

Function	<code>RStringF AddressType() const</code>
Description	Gets the address type.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The address type from predefined SDP string table or entered by the user.
Postconditions	None
Exceptions	-

2.4.2.19 InetAddress

Function	<code>const TInetAddr* InetAddress() const</code>
Description	Gets the address.
Preconditions	None
Parameters	None
Return value	<code>const TInetAddr*</code> : Address as an IP address or null if it is not an IP address. This may be, for example, when the address has been specified as an FQDN. In this case, the address can be accessed using the other getters.
Postconditions	None
Exceptions	-

2.4.2.20 Address

Function	<code>const TDesC8& Address() const</code>
Description	Gets the address.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : Address in a string format.
Postconditions	None
Exceptions	-

2.4.2.21 SetInetAddress

Function	<code>void SetInetAddress(const TInetAddr& aValue)</code>
Description	Sets the address, network, and address type. Also sets the network type to "IN" and address type to "IP4" or "IP6" depending on the address family of <code>aValue</code> .
Preconditions	None
Parameters	<code>const TInetAddr& aValue</code> : A unicast IP address from either <code>KAfInet</code> or <code>KAfInet6</code> family.
Return value	None
Postconditions	None
Exceptions	-

2.4.2.2 SetAddressL

Function	<code>void SetAddressL(const TDesC8& aAddress, RStringF aNetType, RStringF aAddressType)</code>
Description	Sets the address, network, and address type.
Preconditions	None
Parameters	<code>const TDesC8& aAddress</code> : A unicast IP Address, Fully Qualified Domain Name (FQDN), or another allowed type of address. <code>RStringF aNetType</code> : A valid network type value (typically "IN"). <code>RStringF aAddressType</code> : A valid address type value (typically "IP4" or "IP6").
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecOriginField</code> if <code>aNetType</code> or <code>aAddressType</code> are not valid tokens, <code>aAddress</code> is invalid, or <code>aAddress</code> type is not aligned with the address format.

2.4.3 CSdpConnectionField

This class encapsulates the connection information field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.3.1 DecodeL

Function	<code>static CSdpConnectionField* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new connection field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpConnectionField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.2 DecodeLC

Function	<code>static CSdpConnectionField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new connection field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpConnectionField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.3 NewL

Function	<code>static CSdpConnectionField* NewL(const TInetAddr& aAddress, TInt aTTL = KErrNotFound, TInt aNumOfAddress = KDefaultNumOfAddress)</code>
Description	Constructs a new connection field. Also sets the network type to "IN" and address type to "IP4" or "IP6" depending on the address family of <code>aAddress</code> .
Preconditions	None
Parameters	<code>const TInetAddr& aAddress</code> : IP address from either <code>KAfInet</code> or <code>KAfInet6</code> family. <code>TInt aTTL</code> : Time-To-Live for IP4 multicasts, set it as <code>KErrNotFound</code> in case of IPv6 multicast or IPv4 unicast. <code>TInt aNumOfAddress</code> : Number of addresses in multicast. Must be 1 if unicast.
Return value	<code>CSdpConnectionField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.4 NewL

Function	<code>static CSdpConnectionField* NewL(RStringF aNetType, RStringF aAddressType, const TDesC8& aAddress)</code>
Description	Constructs a new connection field.
Preconditions	None
Parameters	RStringF aNetType: A valid network type name from the predefined SDP string table or defined by the user. RStringF aAddressType: A valid address type name from the predefined SDP string table or defined by the user. const TDesC8& aAddress: A valid address of the address type. Can contain TTL and number of addresses parameters as well.
Return value	CSdpConnectionField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.5 NewLC

Function	<code>static CSdpConnectionField* NewLC(const TInetAddr& aAddress, TInt aTTL = KErrNotFound, TUint aNumOfAddress = KDefaultNumOfAddress)</code>
Description	Constructs a new connection field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	const TInetAddr& aAddress: IP address from either KAfInet or KAfInet6 family. TInt aTTL: Time-To-Live for IP4 multicasts. Set to KErrNotFound in case of IPv6 multicast or IPv4 unicast. TUint aNumOfAddress: Number of addresses in multicast. Must be 1 if unicast.
Return value	CSdpConnectionField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.6 *NewLC*

Function	<code>static CSdpConnectionField* NewLC(RStringF aNetType, RStringF aAddressType, const TDesC8& aAddress)</code>
Description	Constructs a new connection field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	RStringF aNetType: A valid network type name from the predefined SDP string table or defined by the user. RStringF aAddressType: A valid address type name from the predefined SDP string table or defined by the user. const TDesC8& aAddress: A valid address of the address type. Can contain TTL and number of addresses parameters as well.
Return value	CSdpConnectionField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.7 *~CSdpConnectionField*

Function	<code>~CSdpConnectionField()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.3.8 *EncodeL*

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the field formatted according to the SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	RWriteStream& aStream: Stream used for output. On return, the stream includes a correctly formatted connection field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.9 CloneL

Function	<code>CSdpConnectionField* CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpConnectionField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.10 operator==

Function	<code>TBool operator== (const CSdpConnectionField& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpConnectionField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.3.11 NetType

Function	<code>RStringF NetType() const</code>
Description	Gets the network type from the predefined SDP string table or entered by the user.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The network type.
Postconditions	None
Exceptions	-

2.4.3.12 AddressType

Function	<code>RStringF AddressType() const</code>
Description	Gets the address type that is from the predefined SDP string table or entered by the user.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The address type.
Postconditions	None
Exceptions	-

2.4.3.13 InetAddress

Function	<code>const TInetAddr* InetAddress() const</code>
Description	Gets the address.
Preconditions	None
Parameters	None
Return value	<code>const TInetAddr*</code> : Address as an IP address or null if it is not an IP address. This may be, for example, when the address has been specified as an FQDN. In this case, the address can be accessed using the other getters.
Postconditions	None
Exceptions	-

2.4.3.14 Address

Function	<code>const TDesC8& Address() const</code>
Description	Gets the address.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : Address as a string.
Postconditions	None
Exceptions	-

2.4.3.15 *SetInetAddressL*

Function	<code>void SetInetAddressL(const TInetAddr& aValue, TInt aTTL = KErrNotFound, TInt aNumOfAddress = KDefaultNumOfAddress)</code>
Description	Sets the address, network, and address type. Also sets the network type to "IN" and address type to "IP4" or "IP6" depending on the address family of aValue.
Preconditions	None
Parameters	<code>const TInetAddr& aValue</code> : The new address. <code>TInt aTTL</code> : Time-To-Live for IP4 multicasts, set it as KErrNotFound in case of IPv6 multicast or IPv4 unicast. <code>TInt aNumOfAddress</code> : Number of addresses in multicast. Must be 1 if unicast.
Return value	None
Postconditions	None
Exceptions	KErrSdpCodecConnectionField if address to be set is wrongly formatted.

2.4.3.16 *SetAddressL*

Function	<code>void SetAddressL(RStringF aNetType, RStringF aAddressType, const TDesC8& aAddress)</code>
Description	Sets the address, network, and address type.
Preconditions	None
Parameters	<code>RStringF aNetType</code> : A valid network type name from the predefined SDP string table or defined by the user. <code>RStringF aAddressType</code> : A valid address type name from the predefined SDP string table or defined by the user. <code>const TDesC8& aAddress</code> : A valid address of the address type.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.3.17 TTLValue

Function	<code>TInt TTLValue() const</code>
Description	Gets TTL attribute.
Preconditions	None
Parameters	None
Return value	<code>TInt</code> : TTL or <code>KErrNotFound</code> if one is not available for the address.
Postconditions	None
Exceptions	-

2.4.3.18 SetTTL

Function	<code>void SetTTL(TInt aTTL)</code>
Description	Sets TTL attribute (only valid for IP4 multicasts). Leaves if trying to set TTL to an address that doesn't support it.
Preconditions	None
Parameters	<code>TInt aTTL</code> : Time-To-Live for IP4 multicasts.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecConnectionField</code> if <code>aTTL</code> is invalid.

2.4.3.19 NumOfAddress

Function	<code>TInt NumOfAddress() const</code>
Description	Gets the number of addresses (can be more than 1 for multicasts). Multicast addresses are contiguously allocated above the base address.
Preconditions	None
Parameters	None
Return value	<code>TInt</code> : Number of addresses.
Postconditions	None
Exceptions	-

2.4.3.20 SetNumOfAddressL

Function	<code>void SetNumOfAddressL(TInt aNumOfAddress)</code>
Description	Sets the number of addresses allocated for multicast.
Preconditions	None
Parameters	<code>TInt aNumOfAddress</code> : Number of addresses in multicast.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecConnectionField</code> . Leaves if the address is unicast.

2.4.4 CSdpBandwidthField

This class encapsulates the bandwidth field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.4.1 DecodeL

Function	<code>static CSdpBandwidthField* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new bandwidth field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpBandwidthField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.2 DecodeLC

Function	<code>static CSdpBandwidthField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new bandwidth field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpBandwidthField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.3 *NewL*

Function	<code>static CSdpBandwidthField* NewL(RStringF aModifier, TUint32 aValue)</code>
Description	Constructs a new bandwidth field.
Preconditions	None
Parameters	RStringF aModifier: Bandwidth modifier. TUint32 aValue: Bandwidth value.
Return value	CSdpBandwidthField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.4 *NewLC*

Function	<code>static CSdpBandwidthField* NewLC(RStringF aModifier, TUint32 aValue)</code>
Description	Constructs a new bandwidth field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	RStringF aModifier: Bandwidth modifier. TUint32 aValue: Bandwidth value.
Return value	CSdpBandwidthField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.5 *~CSdpBandwidthField*

Function	<code>~CSdpBandwidthField ()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.4.6 *EncodeL*

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Output the field formatted according to SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted bandwidth field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.7 *CloneL*

Function	<code>CSdpBandwidthField * CloneL() const</code>
Description	Creates a new instance that is equal with the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpBandwidthField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.4.8 *operator==*

Function	<code>TBool operator == (const CSdpBandwidthField& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpBandwidthField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.4.9 Modifier

Function	<code>RStringF Modifier() const</code>
Description	Gets the bandwidth modifier.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The modifier that can be predefined in the SDP string table or defined by the user.
Postconditions	None
Exceptions	-

2.4.4.10 SetModifier

Function	<code>void SetModifier(RStringF aModifier)</code>
Description	Sets the bandwidth modifier.
Preconditions	None
Parameters	<code>RStringF aModifier</code> : A valid bandwidth modifier name.
Return value	None
Postconditions	None
Exceptions	-

2.4.4.11 Value

Function	<code>TUint32 Value() const</code>
Description	Gets the bandwidth value.
Preconditions	None
Parameters	None
Return value	<code>TUint32</code> : The value.
Postconditions	None
Exceptions	-

2.4.4.12 SetValue

Function	<code>void SetValue(TUint32 aValue)</code>
Description	Sets the bandwidth value.
Preconditions	None
Parameters	<code>TUint32 aValue</code> : The value.
Return value	None
Postconditions	None
Exceptions	-

2.4.5 CSdpTimeField

This class encapsulates the time field and the related zero or more repeat times fields and an optional time zone adjustment field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.5.1 DecodeL

Function	<code>static CSdpTimeField * DecodeL(const TDesC8& aText, TBool aRecurse = ETrue)</code>
Description	Constructs a time field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A correctly formatted time field value terminated by a CRLF followed by zero or more repeat times and zero or one time zone adjustment fields separated by a CRLF. <code>TBool aRecurse</code> : Flag to specify whether to decode subfields as well (ETrue) or only the time field (EFalse).
Return value	<code>CSdpTimeField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.2 *DecodeLC*

Function	<code>static CSdpTimeField * DecodeLC(const TDesC8& aText, TBool aRecurse = ETrue)</code>
Description	Constructs a time field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A correctly formatted time field value terminated by a CRLF followed by zero or more repeat times and zero or one time zone adjustment fields separated by a CRLF. <code>TBool aRecurse</code> : Flag to specify whether to decode subfields as well (ETrue) or only the time field (EFalse).
Return value	<code>CSdpTimeField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.3 *NewL*

Function	<code>static CSdpTimeField * NewL(const TDesC8& aStartTime, const TDesC8& aStopTime)</code>
Description	Constructs a time field.
Preconditions	None
Parameters	<code>const TDesC8& aStartTime</code> : Session start time. <code>const TDesC8& aStopTime</code> : Session end time that must be greater than or equal to the start time.
Return value	<code>CSdpTimeField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.4 *NewLC*

Function	<code>static CSdpTimeField * NewLC(const TDesC8& aStartTime, const TDesC8& aStopTime)</code>
Description	Constructs a time field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aStartTime</code> : Session start time. <code>const TDesC8& aStopTime</code> : Session end time that must be greater than or equal to the start time.
Return value	<code>CSdpTimeField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.5 *~CsdpTimeField*

Function	<code>~CSdpTimeField()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.5.6 *EncodeL*

Function	<code>void EncodeL(RWriteStream& aStream, TBool aRecurse = ETrue) const</code>
Description	Outputs the field formatted according to the SDP syntax and including the terminating CRLF. Optionally outputs also the related repeat times.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted time field with repeat fields if <code>aRecurse</code> is <code>ETrue</code> . <code>TBool aRecurse</code> : Flag to specify whether to output subfields as well (<code>ETrue</code>) or only the time field (<code>EFalse</code>).
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.7 CloneL

Function	<code>CSdpTimeField * CloneL(TBool aRecurse = ETrue) const</code>
Description	Creates a new instance that is equal to the target. Optionally also related repeat times are cloned.
Preconditions	None
Parameters	<code>TBool aRecurse</code> : Flag to specify whether to clone subfields as well (<code>ETrue</code>) or only the time field (<code>EFalse</code>).
Return value	<code>CSdpTimeField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.5.8 operator==

Function	<code>TBool operator == (const CSdpTimeField & aObj) const</code>
Description	Compares this instance to another to ensure equality. Subfields are included in the comparison if there are any.
Preconditions	None
Parameters	<code>const CSdpTimeField & aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if time description is equal to the other instance and <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.5.9 IsValid

Function	<code>TBool IsValid() const</code>
Description	Checks the consistency of the time description and its subfields.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if the time description is valid and <code>EFalse</code> if it is not.
Postconditions	None
Exceptions	-

2.4.5.10 StartTime

Function	<code>const TDesC8& StartTime() const</code>
Description	Gets the session start time.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : Session start time.
Postconditions	None
Exceptions	-

2.4.5.11 StopTime

Function	<code>const TDesC8& StopTime() const</code>
Description	Gets the session stop time.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : Session stop time.
Postconditions	None
	-

2.4.5.12 SetTimesL

Function	<code>void SetTimesL(const TDesC8& aStartTime, const TDesC8& aStopTime)</code>
Description	Sets the session start and stop times.
Preconditions	None
Parameters	<code>const TDesC8& aStartTime</code> : Session start time. <code>const TDesC8& aStopTime</code> : Session end time. Must be greater than or equal to the start time.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecTimeField</code> if start time and stop time are not correct (as defined [1]).

2.4.5.13 RepeatFields

Function	<code>RPointerArray<CSdpRepeatField>& RepeatFields()</code>
Description	Gets the set of repeat times fields related to this time field. This array is used directly for element insertion and removal. The objects referenced from the array are owned by the media field instance and must not be deleted. An object can be removed from the media description by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpRepeatField>&</code> : Set of repeat fields.
Postconditions	None
Exceptions	-

2.4.6 TSdpTypedTime

This class implements the typed time element of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.6.1 TsdpTypedTime

Function	<code>TSdpTypedTime()</code>
Description	Constructs a typed time equal to zero and with no time unit set.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.6.2 *TsdpTypedTime*

Function	<code>TsdpTypedTime(TInt64 aValue, TUnit aUnit)</code>
Description	Constructs a typed time equal to the specified value and time unit.
Preconditions	None
Parameters	<code>TInt64 aValue</code> : Time value. <code>TUnit aUnit</code> : Time unit.
Return value	None
Postconditions	None
Exceptions	-

2.4.6.3 *operator==*

Function	<code>TBool operator == (const TsdpTypedTime& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const TsdpTypedTime& aObj</code> : Another instance to compare with.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.6.4 *SetValue*

Function	<code>void SetValue(TInt64 aValue, TUnit aUnit)</code>
Description	Sets the typed time value.
Preconditions	none
Parameters	<code>TInt64 aValue</code> : Time value. <code>TUnit aUnit</code> : Time unit.
Return value	The new instance.
Postconditions	None
Exceptions	-

2.4.7 *CSdpRepeatField*

This class encapsulates the repeat times field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.7.1 *DecodeL*

Function	<code>static CSdpRepeatField* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new repeat times field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpRepeatField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.2 *DecodeLC*

Function	<code>static CSdpRepeatField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new repeat times field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpRepeatField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.3 *NewL*

Function	<code>static CSdpRepeatField* NewL(const TSdpTypedTime aRepeatInterval, const TSdpTypedTime aActiveDuration, const RArray<TSdpTypedTime>& aOffsets);</code>
Description	Constructs a new repeat times field.
Preconditions	None
Parameters	<code>const TSdpTypedTime aRepeatInterval</code> : Repeat interval value, greater than zero. <code>const TSdpTypedTime aActiveDuration</code> : Active duration. <code>const RArray<TSdpTypedTime>& aOffsets</code> : At least one time offset.
Return value	<code>CSdpRepeatField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.4 *NewLC*

Function	<code>static CSdpRepeatField* NewLC(const TSdpTypedTime aRepeatInterval, const TSdpTypedTime aActiveDuration, const RArray<TSdpTypedTime>& aOffsets);</code>
Description	Constructs a new repeat times field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TSdpTypedTime aRepeatInterval</code> : Repeat interval value, greater than zero. <code>const TSdpTypedTime aActiveDuration</code> : Active duration. <code>const RArray<TSdpTypedTime>& aOffsets</code> : At least one time offset.
Return value	<code>CSdpRepeatField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.5 *~CSdpRepeatField*

Function	<code>~CSdpRepeatField()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.7.6 *EncodeL*

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the field formatted according to SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted repeat field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.7 *CloneL*

Function	<code>CSdpRepeatField * CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpRepeatField *</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.7.8 *operator==*

Function	<code>TBool operator == (const CSdpRepeatField & aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpRepeatField & aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.7.9 *RepeatInterval*

Function	<code>const TSdpTypedTime RepeatInterval() const</code>
Description	Gets the repeat interval.
Preconditions	None
Parameters	None
Return value	<code>const TSdpTypedTime</code> : The repeat interval.
Postconditions	None
Exceptions	-

2.4.7.10 SetRepeatIntervalL

Function	<code>void SetRepeatIntervalL(const TSdpTypedTime aValue)</code>
Description	Sets the repeat interval.
Preconditions	None
Parameters	<code>const TSdpTypedTime aValue</code> : Repeat interval value, greater than zero.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecRepeatField</code> if the time value is not a positive integer value.

2.4.7.11 ActiveDuration

Function	<code>const TSdpTypedTime ActiveDuration() const</code>
Description	Gets the active duration.
Preconditions	None
Parameters	None
Return value	<code>const TSdpTypedTime</code> : The active duration.
Postconditions	None
Exceptions	-

2.4.7.12 SetActiveDuration

Function	<code>void SetActiveDuration(const TSdpTypedTime aValue)</code>
Description	Sets the active duration.
Preconditions	None
Parameters	<code>const TSdpTypedTime aValue</code> : Active duration.
Return value	None
Postconditions	None
Exceptions	-

2.4.7.13 TimeOffsets

Function	<code>const RArray<TSdpTypedTime>& TimeOffsets() const</code>
Description	Gets the array of time offsets. This array is used directly for element insertion and removal.
Preconditions	None
Parameters	None
Return value	<code>const RArray<TSdpTypedTime>&</code> : The time offsets.
Postconditions	None
Exceptions	-

2.4.7.14 SetTimeOffsetsL

Function	<code>void SetTimeOffsetsL(const RArray<TSdpTypedTime>& aValue)</code>
Description	Sets the time offsets.
Preconditions	None
Parameters	<code>const RArray<TSdpTypedTime>& aValue</code> : At least one time offset.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8 CSdpKeyField

This class encapsulates the encryption key field of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.8.1 DecodeL

Function	<code>static CSdpKeyField* DecodeL(const TDesC8& aFieldValue)</code>
Description	Constructs a new encryption key field.
Preconditions	None
Parameters	<code>const TDesC8& aFieldValue</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpKeyField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.2 *DecodeLC*

Function	<code>static CSdpKeyField* DecodeLC(const TDesC8& aFieldValue)</code>
Description	Constructs a new encryption key field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aFieldValue</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpKeyField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.3 *NewL*

Function	<code>static CSdpKeyField* NewL(RStringF aMethod, const TDesC8& aEncryptionKey)</code>
Description	Constructs a new encryption key field.
Preconditions	None
Parameters	<code>RStringF aMethod</code> : Encryption method from the predefined SDP string table. User-defined values are not accepted. <code>const TDesC8& aEncryptionKey</code> : A valid encryption key value or an empty descriptor to omit the key part.
Return value	<code>CSdpKeyField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.4 *NewLC*

Function	<code>static CSdpKeyField* NewLC(RStringF aMethod, const TDesC8& aEncryptionKey)</code>
Description	Constructs a new encryption key field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>RStringF aMethod</code> : Encryption method from the predefined SDP string table. User-defined values are not accepted. <code>const TDesC8& aEncryptionKey</code> : A valid encryption key value or an empty descriptor to omit the key part.
Return value	<code>CSdpKeyField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.5 ~CsdpKeyField

Function	<code>~CsdpKeyField()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.8.6 EncodeL

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Writes the instance as a complete SDP field encoded as UTF-8 and formatted as defined in [1].
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted key field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.7 CloneL

Function	<code>CSdpKeyField * CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpKeyField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.8.8 operator==

Function	<code>TBool operator == (const CSdpKeyField& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpKeyField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.8.9 Method

Function	<code>RStringF Method() const</code>
Description	Gets the encryption method.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The method.
Postconditions	None
Exceptions	-

2.4.8.10 EncryptionKey

Function	<code>const TDesC8& EncryptionKey() const</code>
Description	Gets the encryption key.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The key or an empty descriptor if there is no key part.
Postconditions	None
Exceptions	-

2.4.8.11 SetL

Function	<code>void SetL(RStringF aMethod, const TDesC8& aEncryptionKey)</code>
Description	Sets the encryption method and key.
Preconditions	None
Parameters	<code>RStringF aMethod</code> : Encryption method from the predefined SDP string table. User-defined values are not accepted. <code>const TDesC8& aEncryptionKey</code> : A valid encryption key value or an empty descriptor to omit the key part.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9 CSdpAttributeField

This class encapsulates the attribute field of Session Description Protocol for media and session level attributes. Media format/payload type level attributes are supported by the `CSdpFmtAttributeField` class. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.9.1 DecodeL

Function	<code>static CSdpAttributeField* DecodeL(const TDesC8& aText)</code>
Description	Constructs an attribute field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.2 DecodeLC

Function	<code>static CSdpAttributeField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new attribute field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.3 NewL

Function	<code>static CSdpAttributeField * NewL(RStringF aAttribute, const TDesC8& aValue)</code>
Description	Constructs a new attribute field.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : The attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	<code>CSdpAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.4 NewLC

Function	<code>static CSdpAttributeField * NewLC(RStringF aAttribute, const TDesC8& aValue)</code>
Description	Constructs a new attribute field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : The attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	<code>CSdpAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.5 ~CsdpAttributeField

Function	<code>~ CSdpAttributeField ()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.9.6 EncodeL

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the field formatted according to the SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted attribute field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.7 CloneL

Function	<code>CSdpAttributeField* CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.9.8 operator==

Function	<code>TBool operator == (const CSdpAttributeField& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpAttributeField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.9.9 Attribute

Function	<code>RStringF Attribute () const</code>
Description	Gets the attribute name.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The attribute name. The name can be predefined in the SDP string table or defined by the client.
Postconditions	None
Exceptions	-

2.4.9.10 Value

Function	<code>const TDesC8& Value () const</code>
Description	Gets the attribute.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The value or an empty descriptor if there is no value part.
Postconditions	None
Exceptions	-

2.4.9.11 SetL

Function	<code>void SetL(RStringF aAttribute, const TDesC8& aValue)</code>
Description	Sets the attribute and attribute value.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : A valid attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaAttributeField</code> if the attribute part is not predefined <code>EAttributeFmt</code> or <code>EAttributeRtpmap</code> .

2.4.10 CSdpFmtAttributeField

This class encapsulates the attribute field of the Session Description Protocol for media format/payload type level attributes. Media and session level attributes are supported by the `CSdpAttributeField` class. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.10.1 DecodeL

Function	<code>static CSdpFmtAttributeField* DecodeL(const TDesC8& aText)</code>
Description	Constructs a new media format level attribute field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpFmtAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.2 DecodeLC

Function	<code>static CSdpFmtAttributeField* DecodeLC(const TDesC8& aText)</code>
Description	Constructs a new media format level attribute field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A string containing a correctly formatted field value terminated by a CRLF.
Return value	<code>CSdpFmtAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.3 NewL

Function	<code>static CSdpFmtAttributeField* NewL(RStringF aAttribute , const TDesC8& aFormat, const TDesC8& aValue)</code>
Description	Constructs a new media level format attribute field.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : The attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aFormat</code> : A valid format/payload identifier used in the format list of the media description. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	<code>CSdpFmtAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.4 NewLC

Function	<code>static CSdpFmtAttributeField* NewLC(RStringF aAttribute , const TDesC8& aFormat, const TDesC8& aValue)</code>
Description	Constructs a new media format level attribute field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : The attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aFormat</code> : A valid format/payload identifier used in the format list of the media description. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	<code>CSdpFmtAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.5 ~CSdpFmtAttributeField

Function	<code>~CSdpFmtAttributeField ()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.10.6 EncodeL

Function	<code>void EncodeL(RWriteStream& aStream) const</code>
Description	Outputs the field formatted according to SDP syntax and including the terminating CRLF.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted attribute field.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.7 CloneL

Function	<code>CSdpFmtAttributeField* CloneL() const</code>
Description	Creates a new instance that is equal to the target.
Preconditions	None
Parameters	None
Return value	<code>CSdpFmtAttributeField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.10.8 operator==

Function	<code>TBool operator == (const CSdpFmtAttributeField& aObj) const</code>
Description	Compares this instance to another to ensure equality.
Preconditions	None
Parameters	<code>const CSdpFmtAttributeField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.10.9 Attribute

Function	<code>RStringF Attribute () const</code>
Description	Gets the attribute name.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The attribute name. The name can be predefined in the SDP string table or defined by the client.
Postconditions	None
Exceptions	-

2.4.10.10 *Format*

Function	<code>const TDesC8& Format () const</code>
Description	Gets the format.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The format.
Postconditions	None
Exceptions	-

2.4.10.11 *Value*

Function	<code>const TDesC8& Value () const</code>
Description	Gets the attribute value.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The value or an empty descriptor if there is no value part.
Postconditions	None
Exceptions	-

2.4.10.12 *SetL*

Function	<code>void SetL(RStringF aAttribute , const TDesC8& aFormat, const TDesC8& aValue)</code>
Description	Sets the attribute, format/payload type, and attribute value.
Preconditions	None
Parameters	<code>RStringF aAttribute</code> : A valid attribute name. It can be a predefined name in the SDP string pool or defined by the client. <code>const TDesC8& aFormat</code> : A valid format/payload identifier used in the format list of the media description. <code>const TDesC8& aValue</code> : A valid attribute value or an empty descriptor to omit the value part.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaAttributeField</code> if <code>aAttribute</code> is not <code>rtpmap</code> or <code>fntp</code> .

2.4.11 *TSdpRtpmapValue*

This class is a utility class for manipulating the value of an `rtpmap` attribute.

The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.11.1 *TsdpRtpmapValue*

Function	<code>TSdpRtpmapValue(const TPtrC8& aEncodingName, const TPtrC8& aClockrate, const TPtrC8& aEncodingParameters)</code>
Description	Initializes the instance to refer to the given <code>rtpmap</code> components. The parameters must stay in scope as long as this instance exists. Otherwise, the member variables of this class will point to a released memory.
Preconditions	None
Parameters	<code>const TPtrC8& aEncodingName</code> : Valid <code>rtpmap</code> encoding name value. <code>const TPtrC8& aClockrate</code> : Valid <code>rtpmap</code> clockrate value. <code>const TPtrC8& aEncodingParameters</code> : Valid <code>rtpmap</code> encoding parameter list.
Return value	None
Postconditions	None
Exceptions	-

2.4.11.2 *DecodeL*

Function	<code>static TSdpRtpmapValue DecodeL(const TDesC8& aText)</code>
Description	Tries to parse a string as an <code>rtpmap</code> attribute value up to the end of the string or CRLF, whichever comes first, and set pointers to the components found. The parameter must stay in scope as long as this instance exists. Otherwise, the member variables of this class will point to a released memory.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : String containing the value of an <code>rtpmap</code> attribute.
Return value	<code>TSdpRtpmapValue</code> : The <code>rtpmap</code> attribute with value.
Postconditions	None
Exceptions	<code>KErrArgument</code> if parsing fails.

2.4.11.3 EncodeL

Function	<code>HBufC8* EncodeL() const</code>
Description	Outputs the attribute value formatted according to the SDP syntax. The field terminating CRLF is not included in the output. When constructing <code>CSdpFmtAttributeField</code> , the return value of this function can be used as <code>aValue</code> in the <code>CSdpFmtAttributeField::NewL(RStringF aAttribute, const TDesC8& aFormat, const TDesC8& aValue)</code> function.
Preconditions	None
Parameters	None
Return value	<code>HBufC8*</code> : Encoded <code>TSdpRtpmapValue</code> . Ownership of the buffer is given to the caller.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12 CSdpMediaField

This class encapsulates the media description field and related media-specific subfields of the Session Description Protocol. The normative reference for correct formatting and values is [1] unless specified otherwise in member documentation. The implementation supports this normative reference, but does not fully enforce it.

2.4.12.1 DecodeL

Function	<code>static CSdpMediaField* DecodeL(const TDesC8& aText, TBool aRecurse = ETrue)</code>
Description	Constructs a new media description field.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A correctly formatted media field terminated by a CRLF, followed by zero or more media attribute fields that are separated by a CRLF. <code>TBool aRecurse</code> : If <code>ETrue</code> , attempts to construct also media attributes following the CRLF that terminastes the media field.
Return value	<code>CSdpMediaField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.2 DecodeLC

Function	<code>static CSdpMediaField* DecodeLC(const TDesC8& aText, TBool aRecurse = ETrue)</code>
Description	Constructs a new media description field and adds the pointer to the cleanup stack.
Preconditions	None
Parameters	<code>const TDesC8& aText</code> : A correctly formatted media field terminated by a CRLF followed by zero or more media attribute fields that are separated by a CRLF. <code>TBool aRecurse</code> : If <code>ETrue</code> , attempts to construct also media attributes following the CRLF that terminates the media field.
Return value	<code>CSdpMediaField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.3 NewL

Function	<code>static CSdpMediaField* NewL(RStringF aMedia, TUint aPort, RStringF aProtocol, const TDesC8& aFormatList)</code>
Description	Constructs a new media description key field. The optional portcount component of the field is initialized to 1.
Preconditions	None
Parameters	<code>RStringF aMedia</code> : A valid media type name. <code>TUint aPort</code> : Port number. <code>RStringF aProtocol</code> : A valid media protocol name. <code>const TDesC8& aFormatList</code> : A valid format list.
Return value	<code>CSdpMediaField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.4 NewLC

Function	<code>static CSdpMediaField* NewLC(RStringF aMedia, TUint aPort, RStringF aProtocol, const TDesC8& aFormatList)</code>
Description	Constructs a media description field and adds the pointer to the cleanup stack. The optional portcount component of the field is initialized to 1.
Preconditions	None
Parameters	RStringF aMedia: A valid media type name. TUint aPort: Port number. RStringF aProtocol: A valid media protocol name. const TDesC8& aFormatList: A valid format list.
Return value	CSdpMediaField*: The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.5 ~CSdpMediaField

Function	<code>~CSdpMediaField ()</code>
Description	Deletes the resources held by the instance.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.12.6 EncodeL

Function	<code>void EncodeL(RWriteStream& aStream, TBool aRecurse = ETrue) const</code>
Description	Outputs the field formatted according to the SDP syntax and including the terminating CRLF. Optionally outputs also the related media level fields.
Preconditions	None
Parameters	<code>RWriteStream& aStream</code> : Stream used for output. On return, the stream includes the correctly formatted media field with media part fields if <code>aRecurse</code> is defined to <code>ETrue</code> . <code>TBool aRecurse</code> : Flag to specify whether to output also media attributes (<code>ETrue</code>) or only the media field (<code>EFalse</code>).
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.7 CloneL

Function	<code>CSdpMediaField* CloneL(TBool aRecurse = ETrue) const</code>
Description	Create a new instance that is equal to the target. Optionally also related subfields are cloned.
Preconditions	None
Parameters	<code>TBool aRecurse</code> : Flag to specify whether to clone also subfields (<code>ETrue</code>) or only the media field (<code>EFalse</code>).
Return value	<code>CSdpMediaField*</code> : The new instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.8 operator==

Function	<code>TBool operator == (const CSdpAttributeField& aObj) const</code>
Description	Compares this instance to another to ensure equality. Subfields are included in the comparison if there are any.
Preconditions	None
Parameters	<code>const CSdpAttributeField& aObj</code> : The instance to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if equal, <code>EFalse</code> if not.
Postconditions	None
Exceptions	-

2.4.12.9 *IsValid*

Function	<code>TBool IsValid() const</code>
Description	Checks the consistency between the media field and its subfields. In particular, this function checks that each format-specific attribute is related to a format in the format list of the media field.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if the media description is consistent and <code>EFalse</code> if it is not.
Postconditions	None
Exceptions	-

2.4.12.10 *Media*

Function	<code>RStringF Media() const</code>
Description	Gets the media type.
Preconditions	None
Parameters	None
Return value	<code>RStringF</code> : The media type.
Postconditions	None
Exceptions	-

2.4.12.11 *SetMediaL*

Function	<code>void SetMediaL(RStringF aMedia)</code>
Description	Sets the media type.
Preconditions	None
Parameters	<code>RStringF aMedia</code> : The media type.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaField</code> if <code>aMedia</code> is not valid character.

2.4.12.12 Protocol

Function	RStringF Protocol() const
Description	Get the media protocol.
Preconditions	None
Parameters	None
Return value	RStringF: The media protocol.
Postconditions	None
Exceptions	-

2.4.12.13 SetProtocolL

Function	void SetProtocolL(RStringF aProtocol)
Description	Sets the media protocol.
Preconditions	None
Parameters	RStringF aProtocol: The media protocol.
Return value	None
Postconditions	None
Exceptions	KErrSdpCodecMediaField if aProtocol containing two elements divided by a slash are not valid tokens.

2.4.12.14 Port

Function	TUint Port() const
Description	Gets the port number.
Preconditions	None
Parameters	None
Return value	TUint: The port number.
Postconditions	None
Exceptions	-

2.4.12.15 *SetPortL*

Function	<code>void SetPortL(TUint aPort)</code>
Description	Sets the port number.
Preconditions	None
Parameters	<code>TUint aPort</code> : The port number.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaField</code> if the port number is not valid as defined in [1].

2.4.12.16 *PortCount*

Function	<code>TUint PortCount() const</code>
Description	Gets the port count.
Preconditions	None
Parameters	None
Return value	<code>TUint</code> : The port count.
Postconditions	None
Exceptions	-

2.4.12.17 *SetPortCountL*

Function	<code>void SetPortCountL(TUint aCount)</code>
Description	Sets the port count.
Preconditions	None
Parameters	<code>TUint aCount</code> : The port count that must be greater than zero.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaField</code> if <code>aCount</code> equals zero.

2.4.12.18 *FormatList*

Function	<code>const TDesC8& FormatList() const</code>
Description	Gets the format list.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The format list.
Postconditions	None
Exceptions	-

2.4.12.19 *SetFormatListL*

Function	<code>void SetFormatListL(const TDesC8& aValue)</code>
Description	Sets the format list. The format list should contain one or more format tokens separated by a single whitespace character.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : A valid format list.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaField</code> if <code>aValue</code> contains invalid tokens.

2.4.12.20 *Info*

Function	<code>const TDesC8& Info() const</code>
Description	Gets the media level info field value.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The value or an empty descriptor if not present.
Postconditions	None
Exceptions	-

2.4.12.21 *SetInfoL*

Function	<code>void SetInfoL(const TDesC8& aValue)</code>
Description	Sets the media level info field value.
Preconditions	None
Parameters	<code>const TDesC8& aValue</code> : A valid info field value.
Return value	None
Postconditions	None
Exceptions	<code>KErrSdpCodecMediaInfoField</code> if <code>aValue</code> is not <code>KNullDesC8</code> or <code>aValue</code> includes invalid byte strings (<code>'\0'</code> , <code>'\n'</code> , <code>'\r'</code>).

2.4.12.22 *Key*

Function	<code>CSdpKeyField* Key()</code>
Description	Gets the media level encryption key field. Note that the ownership is not transferred and the instance must not be deleted by the caller.
Preconditions	None
Parameters	None
Return value	<code>CSdpKeyField*</code> : Encryption key field or null if not present.
Postconditions	None
Exceptions	-

2.4.12.23 *Key*

Function	<code>const CSdpKeyField* Key() const</code>
Description	Gets the media level encryption key field.
Preconditions	None
Parameters	None
Return value	<code>const CSdpKeyField*</code> : Encryption key field or null if not present.
Postconditions	None
Exceptions	-

2.4.12.24 *SetKey*

Function	<code>void SetKey(CSdpKeyField* aObj)</code>
Description	Sets or removes the media level encryption key field.
Preconditions	None
Parameters	<code>CSdpKeyField* aObj</code> : The new key field or null if field is to be removed. Ownership of the referenced object is transferred to the media field instance.
Return value	None
Postconditions	None
Exceptions	-

2.4.12.25 *FormatAttributeFields*

Function	<code>RPointerArray<CSdpFmtAttributeField>& FormatAttributeFields()</code>
Description	Gets the set of media format level attributes. This array is used directly for element insertion and removal. The objects referenced from the array are owned by the media field instance and must not be deleted. An object can be removed from the media description by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpFmtAttributeField>&</code> : The set of media format level attributes.
Postconditions	None
Exceptions	-

2.4.12.26 *AttributeFields*

Function	<code>RPointerArray<CSdpAttributeField>& AttributeFields()</code>
Description	Gets the set of media level format-independent attributes. This array is used directly for element insertion and removal. The objects referenced from the array are owned by the media field instance and must not be deleted. An object can be removed from the media description by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpAttributeField>&</code> : The set of media level attributes.
Postconditions	None
Exceptions	-

2.4.12.27 *BandwidthFields*

Function	<code>RPointerArray<CSdpBandwidthField>& BandwidthFields()</code>
Description	Gets the set of media level bandwidth fields. This array is used directly for element insertion and removal. The objects referenced from the array are owned by the media field instance and must not be deleted. An object can be removed from the media description by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpBandwidthField>&</code> : The set of media level bandwidth fields.
Postconditions	None
Exceptions	-

2.4.12.28 *ConnectionFields*

Function	<code>RPointerArray<CSdpConnectionField>& ConnectionFields()</code>
Description	Gets the set of media level connection fields. This array is used directly for element insertion and removal. The objects referenced from the array are owned by the media field instance and must not be deleted. An object can be removed from the media description by setting the corresponding element to zero. By doing so, the calling party receives ownership of the removed object.
Preconditions	None
Parameters	None
Return value	<code>RPointerArray<CSdpConnectionField>&</code> : The set of media level connection fields.
Postconditions	None
Exceptions	-

2.4.12.29 *RemoveFormatL*

Function	<code>void RemoveFormatL(const TDesC8& aFormat)</code>
Description	Removes a specific format from the media description. The function will remove the named format from the format list if found and then delete all format level attributes of type <code>CSdpFmtAttributeField</code> that are related to the removed format.
Preconditions	None
Parameters	<code>const TDesC8& aFormat</code> : The name of the format to be removed.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.30 *KeepFormatL*

Function	<code>void KeepFormatL(const TDesC8& aFormat)</code>
Description	Removes all formats except one from the media description. The function will set the format list to only contain the one format to keep and then delete all format level attributes that are related to the removed formats. If the format is not found from the format list, it is added there. If there are no format level attributes for the specified format, the format level attribute set will be empty.
Preconditions	None
Parameters	<code>const TDesC8& aFormat</code> : The name of the format to keep after removing all others.
Return value	None
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.4.12.31 *RejectMedia*

Function	<code>void RejectMedia()</code>
Description	Sets this media description into rejected state. The rejected state is defined by the offer/answer model in [5]. The function sets the port number to 0 and removes all formats except one. The remaining format is determined by the implementation.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.13 *CSdpCodecStringPool*

This class defines the string pool for SDP Codec. The client of SDP Codec must call `OpenL()` before invoking any function of SDP Codec classes. The client must close the string pool when finished using SDP Codec classes.

2.4.13.1 *OpenL*

Function	<code>static void OpenL()</code>
Description	Opens SDP Codec string pool.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	<code>KErrAlreadyExists</code> if pool already exists in TLS.

2.4.13.2 *Close*

Function	<code>static void Close()</code>
Description	Closes SDP Codec string pool.
Preconditions	None
Parameters	None
Return value	None
Postconditions	None
Exceptions	-

2.4.13.3 *StringPoolL*

Function	<code>static RStringPool StringPoolL()</code>
Description	Gets the string pool used by SDP Codec.
Preconditions	None
Parameters	None
Return value	<code>RStringPool</code> : A handle of a string pool.
Postconditions	None
Exceptions	<code>KErrSdpCodecStringPool</code> if the pool is not opened.

2.4.13.4 StringTableL

Function	<code>static const TStringTable& StringTableL()</code>
Description	Gets the string table used by SDP Codec.
Preconditions	None
Parameters	None
Return value	<code>const TStringTable&</code> : The string pool table.
Postconditions	None
Exceptions	<code>KErrSdpCodecStringPool</code> if the pool is not opened.

2.4.14 Code Architecture

All services offered by the SDP Codec API are provided as a dynamically linked library called `sdpcodec.dll`. The header files of the API are located in `/epoc32/include` and are the following:

File	Description
<code>SdpAttributeField.h</code>	<code>class CSdpAttributeField</code>
<code>SdpBandwidthField.h</code>	<code>class CSdpBandwidthField</code>
<code>SdpConnectionField.h</code>	<code>class CSdpConnectionField</code>
<code>SdpDocument.h</code>	<code>class CSdpDocument</code>
<code>SdpFmtAttributeField.h</code>	<code>class CSdpFmtAttributeField</code>
<code>SdpKeyField.h</code>	<code>class CSdpKeyField</code>
<code>SdpMediaField.h</code>	<code>class CSdpMediaField</code>
<code>SdpOriginField.h</code>	<code>class CSdpOriginField</code>
<code>SdpRepeatField.h</code>	<code>class CSdpRepeatField</code>
<code>SdpRtpmapValue.h</code>	<code>class TSdpRtpmapValue</code>
<code>SdpTimeField.h</code>	<code>class CSdpTimeField</code>
<code>SdpTypedTime.h</code>	<code>class CSdpTypedTime</code>
<code>SdpCodecConstants.h</code>	Constant values of SDP codec
<code>SdpCodecErr.h</code>	Internal error codes of SDP Codec
<code>SdpCodecStringPool.h</code>	Class <code>CSdpCodecStringPool</code>
<code>SdpCodecStringConstants.h</code>	The SDP Codec's string pool constants

3 Index

3.1 List of Constants and Enums

```

/**
 * Codec encoding errors for SDP fields
 */
const TInt KSdpCodecErrorBase = -17875;
const TInt KErrSdpCodecVersionField = KSdpCodecErrorBase - 1;
const TInt KErrSdpCodecOriginField = KSdpCodecErrorBase - 2;
const TInt KErrSdpCodecSessionField = KSdpCodecErrorBase - 3;
const TInt KErrSdpCodecInfoField = KSdpCodecErrorBase - 4;
const TInt KErrSdpCodecUriField = KSdpCodecErrorBase - 5;
const TInt KErrSdpCodecEmailField = KSdpCodecErrorBase - 6;
const TInt KErrSdpCodecPhoneField = KSdpCodecErrorBase - 7;
const TInt KErrSdpCodecConnectionField = KSdpCodecErrorBase - 8;
const TInt KErrSdpCodecBandwidthField = KSdpCodecErrorBase - 9;
const TInt KErrSdpCodecRepeatField = KSdpCodecErrorBase -
10;
const TInt KErrSdpCodecZoneField = KSdpCodecErrorBase -
11;
const TInt KErrSdpCodecKeyField = KSdpCodecErrorBase -
12;
const TInt KErrSdpCodecAttributeField = KSdpCodecErrorBase -
13;
const TInt KErrSdpCodecMediaField = KSdpCodecErrorBase -
14;
const TInt KErrSdpCodecMediaInfoField = KSdpCodecErrorBase -
15;
const TInt KErrSdpCodecMediaConnectionField = KSdpCodecErrorBase -
16;
const TInt KErrSdpCodecMediaBandwidthField = KSdpCodecErrorBase -
17;
const TInt KErrSdpCodecMediaKeyField = KSdpCodecErrorBase -
18;
const TInt KErrSdpCodecMediaAttributeField = KSdpCodecErrorBase -
19;
const TInt KErrSdpCodecTypedTime = KSdpCodecErrorBase -
20;
const TInt KErrSdpCodecStringPool = KSdpCodecErrorBase -
21;
const TInt KErrSdpCodecDecode = KSdpCodecErrorBase -
22;

/** Predefined values in SDP codec String table */
class SdpCodecStringConstants
{
public:
    enum TStrings
    {
        // SDP field names:
        /** v= */
        EVersion,
        /** o= */
        EOrigin,
        /** s= */
        ESession,
        /** i= */
        EInfo,
        /** u= */
        EUri,
        /** e= */
        EEmail,
        /** p= */
        EPhone,
    }
}

```

```

/** c= */
EConnection,
/** b= */
EBandwidth,
/** z= */
EZone,
/** k= */
EKey,
/** a= */
EAttribute,
/** t= */
ETime,
/** r= */
ERepeat,
/** m= */
EMedia,

// Predefined bandwidth modifiers:

/** CT */
EModifierCT,
/** AS */
EModifierAS,

// Typed time units:

/** d */
ETimeUnitDays,
/** h */
ETimeUnitHours,
/** m */
ETimeUnitMinutes,
/** s */
ETimeUnitSeconds,
/** No typed time unit defined */
ETimeUnitNone,

// Predefined encryption methods:

/** clear */
EMethodClear,
/** base64 */
EMethodBase64,
/** uri */
EMethodUri,
/** prompt */
EMethodPrompt,

// Predefined attributes:

/** cat */
EAttributeCat,
/** keywds */
EAttributeKeywds,
/** tool */
EAttributeTool,
/** ptime */
EAttributePtime,
/** maxptime */
EAttributeMaxptime,
/** recvonly */
EAttributeRecvonly,
/** sendrecv */
EAttributeSendrecv,
/** sendonly */
EAttributeSendonly,
/** inactive */
EAttributeInactive,
/** orient */

```

```

EAttributeOrient,

/** portrait */
EAttributeOrientValuePortrait,
/** landscape */
EAttributeOrientValueLandscape,
/** seascape */
EAttributeOrientValueSeascape,

/** type */
EAttributeType,
/** charset */
EAttributeCharset,
/** sdplang */
EAttributeSdplang,
/** lang */
EAttributeLang,
/** framerate */
EAttributeFramerate,
/** quality */
EAttributeQuality,
/** control */
EAttributeControl,
/** range */
EAttributeRange,
/** etag */
EAttributeEtag,
/** des */
EAttributeDes,
/** curr */
EAttributeCurr,
/** conf */
EAttributeConf,

/** mandatory */
EAttributeStatusStrengthTagMandatory,
/** optional */
EAttributeStatusStrengthTagOptional,
/** none */
EAttributeStatusStrengthTagNone,
/** failure */
EAttributeStatusStrengthTagFailure,
/** unknown */
EAttributeStatusStrengthTagUnknown,

// Predefined Status type for Curr, Conf and Des attributes:

/** e2e */
EAttributeStatusTypeE2e,
/** local */
EAttributeStatusTypeLocal,
/** remote */
EAttributeStatusTypeRemote,

/** none */
EAttributeStatusDirectionTagNone,
/** send */
EAttributeStatusDirectionTagSend,
/** recv */
EAttributeStatusDirectionTagRecv,
/** sendrecv */
EAttributeStatusDirectionTagSendrecv,
/** mid */
EAttributeMid,
/** group */
EAttributeGroup,

// Predefined semantics for Group attribute:

```

```

    /** LS */
    EAttributeGroupSemanticLS,
    /** FID */
    EAttributeGroupSemanticFID,
    /** SRF */
    EAttributeGroupSemanticSRF,
    /** rtpmap */
    EAttributeRtpmap,
    /** fmpdp */
    EAttributeFmpdp,

    // Predefined media types:

    /** audio */
    EMediaAudio,
    /** video */
    EMediaVideo,
    /** application */
    EMediaApplication,
    /** data */
    EMediaData,
    /** control */
    EMediaControl,
    /** image */
    EMediaImage,

    // Predefined media transport protocols:

    /** RTP/AVP */
    EProtocolRtpAvp,
    /** udp */
    EProtocolUdp,
    /** TCP */
    EProtocolTcp,

    // Network type:

    /** IN */
    ENetType,

    // Address types:

    /** IP4 */
    EAddressTypeIP4,
    /** IP6 */
    EAddressType
};

static const TStringTable Table;
};

```

TSdpTypedTime includes the following enum list:

```

enum TUnit
{
    /** d */
    ETimeUnitDays,
    /** h */
    ETimeUnitHours,
    /** m */
    ETimeUnitMinutes,
    /** s */
    ETimeUnitSeconds,
    /** none */
    ETimeUnitNone
};

```


3.2 Alphabetical List of Classes and Methods

3.2.1 CSdpAttributeField

```

IMPORT_C ~CSdpAttributeField();
IMPORT_C RStringF Attribute() const;
IMPORT_C CSdpAttributeField* CloneL() const;
IMPORT_C static CSdpAttributeField* DecodeL( const TDesC8& aText );
IMPORT_C static CSdpAttributeField* DecodeLC( const TDesC8& aText );
IMPORT_C void EncodeL( RWriteStream& aStream ) const;
IMPORT_C static CSdpAttributeField* NewL(RStringF aAttribute,
                                         const TDesC8& aValue );
IMPORT_C static CSdpAttributeField* NewLC(RStringF aAttribute,
                                         const TDesC8& aValue );
IMPORT_C TBool operator == ( const CSdpAttributeField& aObj ) const;
IMPORT_C void SetL( RStringF aAttribute, const TDesC8& aValue );
IMPORT_C const TDesC8& Value() const;

```

3.2.2 CSdpBandwidthField

```

IMPORT_C ~CSdpBandwidthField();
IMPORT_C CSdpBandwidthField * CloneL() const;
IMPORT_C static CSdpBandwidthField* DecodeL(const TDesC8& aText);
IMPORT_C static CSdpBandwidthField* DecodeLC(const TDesC8& aText);
IMPORT_C void EncodeL(RWriteStream& aStream) const;
IMPORT_C RStringF Modifier() const;
IMPORT_C static CSdpBandwidthField* NewL(RStringF aModifier, TUint32
aValue);
IMPORT_C static CSdpBandwidthField* NewLC(RStringF aModifier, TUint32
aValue);
IMPORT_C TBool operator == (const CSdpBandwidthField& aObj) const;
IMPORT_C void SetModifier(RStringF aModifier);
IMPORT_C void SetValue(TUint32 aValue);
IMPORT_C TUint32 Value() const;

```

3.2.3 CSdpCodecStringPool

```

IMPORT_C static void Close();
IMPORT_C static void OpenL();
IMPORT_C static RStringPool StringPoolL();
IMPORT_C static const TStringTable& StringTableL();

```

3.2.4 CSdpConnectionField

```

IMPORT_C ~CSdpConnectionField();
IMPORT_C const TDesC8& Address() const;
IMPORT_C RStringF AddressType() const;
IMPORT_C CSdpConnectionField* CloneL() const;
IMPORT_C static CSdpConnectionField* DecodeL( const TDesC8& aText );
IMPORT_C static CSdpConnectionField* DecodeLC( const TDesC8& aText );
IMPORT_C void EncodeL( RWriteStream& aStream ) const;
IMPORT_C const TInetAddr* InetAddress() const;
IMPORT_C RStringF NetType() const;
IMPORT_C static CSdpConnectionField* NewL(
    const TInetAddr& aAddress,
    TInt aTTL = KErrNotFound,
    TUint aNumOfAddress = KDefaultNumOfAddress );
IMPORT_C static CSdpConnectionField* NewL(RStringF aNetType,
    RStringF aAddressType,
    const TDesC8& aAddress );
IMPORT_C static CSdpConnectionField* NewLC(
    const TInetAddr& aAddress,
    TInt aTTL = KErrNotFound,
    TUint aNumOfAddress = KDefaultNumOfAddress );
IMPORT_C static CSdpConnectionField* NewLC( RStringF aNetType,

```

```

RStringF aAddressType,
const TDesC8& aAddress );
IMPORT_C TInt NumOfAddress() const;
IMPORT_C TBool operator==( const CSdpConnectionField& aObj ) const;
IMPORT_C void SetAddressL( RStringF aNetType,
RStringF aAddressType,
const TDesC8& aAddress );
IMPORT_C void SetInetAddressL( const TInetAddr& aValue,
TInt aTTL = KErrNotFound,
TUint aNumOfAddress =
KDefaultNumOfAddress);
IMPORT_C void SetNumOfAddressL( TUint aNumOfAddress );
IMPORT_C void SetTTLL( TInt aTTL );
IMPORT_C TInt TTLValue() const;

```

3.2.5 CSdpDocument

```

IMPORT_C ~CSdpDocument();
IMPORT_C RPointerArray<CSdpAttributeField>& AttributeFields();
IMPORT_C RPointerArray<CSdpBandwidthField>& BandwidthFields();
IMPORT_C CSdpDocument* CloneL() const;
IMPORT_C CSdpConnectionField* ConnectionField();
IMPORT_C const CSdpConnectionField* ConnectionField() const;
IMPORT_C static CSdpDocument* DecodeL(const TDesC8& aText);
IMPORT_C static CSdpDocument* DecodeLC(const TDesC8& aText);
IMPORT_C CDesC8Array& EmailFields();
IMPORT_C void EncodeL(RWriteStream& aStream) const;
IMPORT_C void ExternalizeL(RWriteStream& aStream) const;
IMPORT_C const TDesC8& Info() const;
IMPORT_C static CSdpDocument* InternalizeL(RReadStream& aStream);
IMPORT_C TBool IsValid() const;
IMPORT_C CSdpKeyField* Key();
IMPORT_C const CSdpKeyField* Key() const;
IMPORT_C RPointerArray<CSdpMediaField>& MediaFields();
IMPORT_C static CSdpDocument* NewL();
IMPORT_C static CSdpDocument* NewLC();
IMPORT_C CSdpOriginField* OriginField();
IMPORT_C const CSdpOriginField* OriginField() const;
IMPORT_C CDesC8Array& PhoneFields();
IMPORT_C TUint SdpVersion() const;
IMPORT_C const TDesC8& SessionName() const;
IMPORT_C void SetConnectionField(CSdpConnectionField* aObj);
IMPORT_C void SetInfoL(const TDesC8& aValue);
IMPORT_C void SetKey(CSdpKeyField* aObj);
IMPORT_C void SetOriginField(CSdpOriginField* aObj);
IMPORT_C void SetSessionNameL(const TDesC8& aValue);
IMPORT_C void SetUri(CUri8* aValue);
IMPORT_C void SetZoneAdjustmentsL(const TDesC8& aValue);
IMPORT_C RPointerArray<CSdpTimeField>& TimeFields();
IMPORT_C CUri8* Uri();
IMPORT_C const CUri8* Uri() const;
IMPORT_C const TDesC8& ZoneAdjustments() const;

```

3.2.6 CSdpFmtAttributeField

```

IMPORT_C ~CSdpFmtAttributeField();

IMPORT_C RStringF Attribute() const;

IMPORT_C CSdpFmtAttributeField * CloneL() const;

IMPORT_C static CSdpFmtAttributeField* DecodeL( const TDesC8& aText );

IMPORT_C static CSdpFmtAttributeField* DecodeLC( const TDesC8& aText);
IMPORT_C void EncodeL( RWriteStream& aStream ) const;
IMPORT_C const TDesC8& Format() const;
IMPORT_C static CSdpFmtAttributeField* NewL( RStringF aAttribute,

```

```

const TDesC8& aFormat,
const TDesC8& aValue );
IMPORT_C static CSdpFmtAttributeField* NewLC(RStringF aAttribute,
const TDesC8& aFormat,
const TDesC8& aValue );
IMPORT_C TBool operator== ( const CSdpFmtAttributeField& aObj ) const;
IMPORT_C void SetL(RStringF aAttribute,
const TDesC8& aFormat,
const TDesC8& aValue );
IMPORT_C const TDesC8& Value() const;

```

3.2.7 CSdpKeyField

```

IMPORT_C ~CSdpKeyField();
IMPORT_C CSdpKeyField * CloneL() const;
IMPORT_C static CSdpKeyField* DecodeL(const TDesC8& aFieldValue);
IMPORT_C static CSdpKeyField* DecodeLC(const TDesC8& aFieldValue);
IMPORT_C void EncodeL(RWriteStream& aStream) const;
IMPORT_C const TDesC8& EncryptionKey() const;
IMPORT_C RStringF Method() const;
IMPORT_C static CSdpKeyField* NewL(RStringF aMethod,
const TDesC8& aEncryptionKey);
IMPORT_C static CSdpKeyField* NewLC(RStringF aMethod,
const TDesC8& aEncryptionKey);
IMPORT_C TBool operator == (const CSdpKeyField& aObj) const;
IMPORT_C void SetL(RStringF aMethod, const TDesC8& aEncryptionKey);

```

3.2.8 CSdpMediaField

```

IMPORT_C ~CSdpMediaField();
IMPORT_C RPointerArray<CSdpAttributeField>& AttributeFields();
IMPORT_C RPointerArray<CSdpBandwidthField>& BandwidthFields();
IMPORT_C CSdpMediaField * CloneL(TBool aRecurse = ETrue) const;
IMPORT_C RPointerArray<CSdpConnectionField>& ConnectionFields();
IMPORT_C static CSdpMediaField* DecodeL(const TDesC8& aText,
TBool aRecurse = ETrue);
IMPORT_C static CSdpMediaField* DecodeLC(const TDesC8& aText,
TBool aRecurse = ETrue);
IMPORT_C void EncodeL(RWriteStream& aStream, TBool aRecurse = ETrue)
const;
IMPORT_C RPointerArray<CSdpFmtAttributeField>& FormatAttributeFields();
IMPORT_C const TDesC8& FormatList() const;
IMPORT_C const TDesC8& Info() const;
IMPORT_C TBool IsValid() const;
IMPORT_C void KeepFormatL(const TDesC8& aFormat);
IMPORT_C CSdpKeyField* Key();
IMPORT_C const CSdpKeyField* Key() const;
IMPORT_C RStringF Media() const;
IMPORT_C static CSdpMediaField* NewL(RStringF aMedia,
TUint aPort,
RStringF aProtocol,
const TDesC8& aFormatList);
IMPORT_C static CSdpMediaField* NewLC(RStringF aMedia,
TUint aPort,
RStringF aProtocol,
const TDesC8& aFormatList);
IMPORT_C TBool operator == (const CSdpMediaField& aObj) const;
IMPORT_C TUint Port() const;
IMPORT_C TUint PortCount() const;
IMPORT_C RStringF Protocol() const;
IMPORT_C void RejectMedia();
IMPORT_C void RemoveFormatL(const TDesC8& aFormat);
IMPORT_C void SetFormatListL(const TDesC8& aValue);
IMPORT_C void SetInfoL(const TDesC8& aValue);
IMPORT_C void SetKey(CSdpKeyField* aObj);
IMPORT_C void SetMediaL(RStringF aMedia);
IMPORT_C void SetPortCountL(TUint aCount);

```

```
IMPORT_C void SetPortL(TUint aPort);
IMPORT_C void SetProtocolL(RStringF aProtocol);
```

3.2.9 CSdpOriginField

```
IMPORT_C ~CSdpOriginField();
IMPORT_C const TDesC8& Address() const;
IMPORT_C RStringF AddressType() const;
IMPORT_C CSdpOriginField* CloneL() const;
IMPORT_C static CSdpOriginField* DecodeL( const TDesC8& aText );
IMPORT_C static CSdpOriginField* DecodeLC( const TDesC8& aText );
IMPORT_C void EncodeL( RWriteStream& aStream ) const;
IMPORT_C const TInetAddr* InetAddress() const;
IMPORT_C RStringF NetType() const;
IMPORT_C static CSdpOriginField* NewL( const TDesC8& aUserName,
                                     TInt64 aSessionId,
                                     TInt64 aSessionVersion,
                                     TInetAddr& aAddress );
IMPORT_C static CSdpOriginField* NewL( const TDesC8& aUserName,
                                     TInt64 aSessionId,
                                     TInt64 aSessionVersion,
                                     RStringF aNetType,
                                     RStringF aAddressType,
                                     const TDesC8& aAddress );
IMPORT_C static CSdpOriginField* NewLC( const TDesC8& aUserName,
                                     TInt64 aSessionId,
                                     TInt64 aSessionVersion,
                                     TInetAddr& aAddress );
IMPORT_C static CSdpOriginField* NewLC( const TDesC8& aUserName,
                                     TInt64 aSessionId,
                                     TInt64 aSessionVersion,
                                     RStringF aNetType,
                                     RStringF aAddressType,
                                     const TDesC8& aAddress );
IMPORT_C TBool operator == ( const CSdpOriginField & aObj ) const;
IMPORT_C TInt64 SessionId() const;
IMPORT_C void SetAddressL( const TDesC8& aAddress,
                          RStringF aNetType,
                          RStringF aAddressType );
IMPORT_C void SetInetAddress( const TInetAddr& aValue );
IMPORT_C void SetSessionId( TInt64 aValue );
IMPORT_C void SetUserNameL( const TDesC8& aValue );
IMPORT_C void SetVersion( TInt64 aValue );
IMPORT_C const TDesC8& UserName() const;
IMPORT_C TInt64 Version() const;
```

3.2.10 CSdpRepeatField

```
IMPORT_C ~CSdpRepeatField();
IMPORT_C const TSdpTypedTime ActiveDuration() const;
IMPORT_C CSdpRepeatField * CloneL() const;
IMPORT_C void EncodeL(RWriteStream& aStream) const;
IMPORT_C static CSdpRepeatField* DecodeL(const TDesC8& aText);
IMPORT_C static CSdpRepeatField* DecodeLC(const TDesC8& aText);
IMPORT_C static CSdpRepeatField* NewL(const TSdpTypedTime
aRepeatInterval,
                                     const TSdpTypedTime
aActiveDuration,
                                     const RArray<TSdpTypedTime>&
aOffsets);
IMPORT_C static CSdpRepeatField* NewLC(const TSdpTypedTime
aRepeatInterval,
                                     const TSdpTypedTime
aActiveDuration,
                                     const RArray<TSdpTypedTime>&
aOffsets);
IMPORT_C TBool operator == (const CSdpRepeatField & aObj) const;
```

```

IMPORT_C const TSdpTypedTime RepeatInterval() const;
IMPORT_C void SetActiveDuration(const TSdpTypedTime aValue);
IMPORT_C void SetRepeatIntervalL(const TSdpTypedTime aValue);
IMPORT_C void SetTimeOffsetsL(const RArray<TSdpTypedTime>& aValue);
IMPORT_C const RArray<TSdpTypedTime>& TimeOffsets() const;

```

3.2.11 CSdpTimeField

```

IMPORT_C ~CSdpTimeField();
IMPORT_C CSdpTimeField * CloneL(TBool aRecurse = ETrue) const;
IMPORT_C void EncodeL(RWriteStream& aStream, TBool aRecurse = ETrue)
const;
IMPORT_C static CSdpTimeField * DecodeL(const TDesC8& aText,
IMPORT_C static CSdpTimeField * DecodeLC(const TDesC8& aText,
                                         TBool aRecurse = ETrue);

IMPORT_C TBool IsValid() const;
IMPORT_C static CSdpTimeField * NewL(const TDesC8& aStartTime,
                                     TBool aRecurse = ETrue);
IMPORT_C static CSdpTimeField * NewLC(const TDesC8& aStartTime,
                                       TBool aRecurse = ETrue);
IMPORT_C TBool operator == (const CSdpTimeField & aObj) const;
IMPORT_C RPointerArray<CSdpRepeatField>& RepeatFields();
IMPORT_C void SetTimesL(const TDesC8& aStartTime, const TDesC8&
aStopTime);
IMPORT_C const TDesC8& StartTime() const;
IMPORT_C const TDesC8& StopTime() const;

```

3.2.12 TSdpRtpmapValue

```

IMPORT_C HBufC8* EncodeL() const;
IMPORT_C static TSdpRtpmapValue DecodeL( const TDesC8& aText );
IMPORT_C TSdpRtpmapValue(const TPtrC8& aEncodingName,
                        const TPtrC8& aClockrate,
                        const TPtrC8& aEncodingParameters );

```

3.2.13 TSdpTypedTime

```

IMPORT_C TBool operator == (const TSdpTypedTime& aObj) const;
IMPORT_C void SetValue(TInt64 aValue, TUnit aUnit);
IMPORT_C TSdpTypedTime();
IMPORT_C TSdpTypedTime(TInt64 aValue, TUnit aUnit);

```

4 Terms and Abbreviations

Term or abbreviation	Meaning
API	Application Programming Interface
FQDN	Fully Qualified Domain Name
MIME	Multipart Internet Mail Extension
RTSP	Real-Time Streaming Protocol
SAP	Session Announcement Protocol
SDK	Software Development Kit
SDP	Session Description Protocol
SIP	Session Initiation Protocol

5 References

- [1] draft-ietf-mmusic-sdp-new-14 <http://www.ietf.org/html.charters/mmusic-charter.html>
- [2] Draft of Real Time Streaming Protocol <http://www.ietf.org/rfc/rfc2326.txt?number=2326>
- [3] Draft of Integration of Resource Management and Session Initiation Protocol
<http://www.ietf.org/rfc/rfc3312.txt?number=3312>
- [4] Draft of Grouping of Media Lines in the Session Description Protocol
<http://www.ietf.org/rfc/rfc3388.txt?number=3388>
- [5] An Offer/Answer Model with the Session Description Protocol (SDP)
<http://www.ietf.org/rfc/rfc3264.txt?number=3264>